Hypergraphes acycliques et résolution de contraintes

Nathan TILLIER

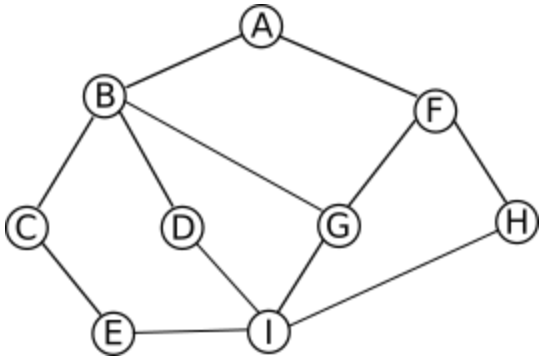
Sommaire

1. Introduction
2. Aspect théorique
3. Algorithmes
4. Implémentation finale
5. Résultats expérimentaux
6. Conclusion

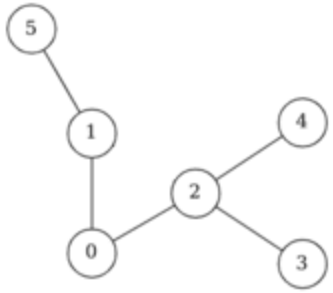
Introduction

Graphes et acyclicité

Graphe cyclique



Graphe acyclique



Liste d'adjacence

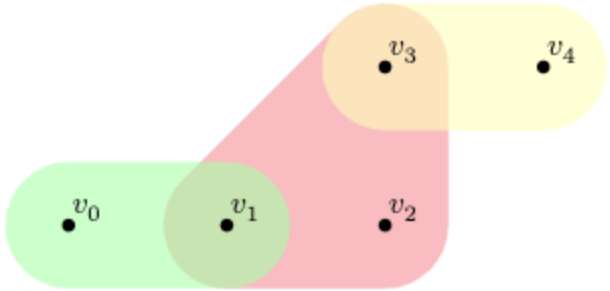
```
A : B, F
B : A, C, D, G
C : B, E
D : B, I
...
```

Liste d'adjacence

```
5 : 1
1 : 0, 5
0 : 1, 2
2 : 0, 3, 4
...
```

Hypergraphes

Hypergraphe



Liste d'adjacence

```
Vert : 0, 1
Rouge : 1, 2, 3
Jaune : 4
```

Définition

Soit $X = \{v_0, v_1, \ldots, v_n\}$ un ensemble fini.
Un hypergraphe sur X est une famille $H = (E_0, E_2, \ldots, E_p)$ de parties de X vérifiant

$$E_i \neq \emptyset \text{ pour } i = 0, 1, \ldots, p$$

Et

$$\bigcup_{i=0}^{p} E_i = X$$

Structure d'hypergraphe

On se munit de fonctions auxiliaires pour manipuler les hypergraphes.

```c
typedef struct arete {
    int cardinal;
    int* sommets;
} arete;

typedef struct hyper {
    int ordre;
    int nb_aretes;
    arete** aretes;
} hyper;
```

```c
arete* creer_arete(int cardinal);
void liberer_arete(arete* a);
arete* copier_arete(arete* a);
void augmenter_arete(arete* a, int sommet);
void reduire_arete(arete* a, int sommet);
bool dans_arete(arete* a, int sommet);

hyper* creer_hyper(int ordre, bool strict);
bool est_valide(hyper* h);
void ajouter_arete(hyper* h, arete* a);
void enlever_arete(hyper* h, int indice);
```

Base de données

On va s'intéresser au schéma suivant :

```
CREATE TABLE communes (
    INT id PRIMARY KEY,
    INT region,
    INT departement,
    CHAR nom
)

CREATE TABLE departements (
    INT id PRIMARY KEY,
    INT region,
    CHAR nom
)

CREATE TABLE regions (
    INT id PRIMARY KEY,
    INT cheflieu,
    CHAR nom
)
```


© Hypergraphes acycliques

Base de données

Communes

Identifiant	Région	Département	Nom
75056	11	75	Paris
29019	53	29	Brest
69123	84	69	Lyon

Régions et départements

Identifiant	Cheflieu	Nom
11	75056	Île-de-France
53	35238	Bretagne
84	69123	Auvergne-Rhône-Alpes

Identifiant	Région	Nom
75	11	Paris
29	53	Finistère
69	84	Rhône

© Hypergraphes acycliques

Aspect théorique

Algèbre relationnelle

$$\text{rel}(a_1 : dom_1, \ldots, a_n : dom_n)$$

Exemple

$$\text{commune}(\text{identifiant} : \text{entier},\ \text{departement} : \text{entier},\ \text{region} : \text{entier},\ \text{nom} : \text{texte})$$

n-uplets

- P = commune(75056, 11, 75, Paris)
- B = commune(29019, 53, 29, Brest)
- L = commune(69123, 84, 69, Lyon)

$$I = \{P, B, L\}$$

© Hypergraphes acycliques

Requête conjonctive

$$q(x_1, \ldots, x_n) : -R_1(\vec{y_1}), \ldots, R_p(\vec{y_p})$$

Exemple

- Schéma

$$\text{commune}(\text{nom} : \text{texte}, \text{departement} : \text{entier})$$

$$\text{departement}(\text{numero} : \text{entier}, \ \text{nom} : \text{texte}, \ \text{region} : \text{texte})$$

- Requête

$$q_1(c, d, r) : -\text{commune}(c, x), \text{departement}(x, d, r)$$

© Hypergraphes acycliques

Arbres de jointure

Définition

Lorem ipsum dolor sit amet.

Algorithmes

Réduction Graham-Yu-Ozsoyoglu

Algorithme de Yannakakis

Implémentation

Résultats expérimentaux

Résultats expérimentaux

Conclusion